Implementation of Chan's Algorithm for Computing the Hypervolume Indicator

Michael Emmerich
University of Jyväskylä, Finland

August 2026

Abstract

The hypervolume indicator, the Lebesgue measure of the region dominated by a set of n points relative to a reference point in $\mathbb{R}^d$, is a special case of Klee's measure problem: computing the volume of a union of n axis-parallel boxes. Chan's FOCS 2013 paper *"Klee's Measure Problem Made Easy"* gives (i) a remarkably simple $\mathcal{O}(n^{d/2})$-time algorithm for arbitrary boxes and (ii) an $\mathcal{O}(n^{d/3}\operatorname{polylog} n)$-time algorithm for orthants—the case that contains the hypervolume indicator problem. We describe complete, self-contained Python implementations of both algorithms; the second supports orthants of arbitrary orientation, with the grounded orthants of hypervolume computation as its primary application. We walk through both algorithms in an implementation-oriented way, including the symbolic machinery of "basic functions" that the $d/3$-algorithm requires, give a transparent complexity analysis of the code as written (including where it loses logarithmic factors against the paper's bounds and why), and validate both implementations exhaustively against independent exact methods. Empirically, the recursion-tree size of the $d/3$-algorithm scales with a visibly smaller exponent than that of the $d/2$-algorithm, while its symbolic bookkeeping carries large constant factors; the simple $d/2$-algorithm remains the practical method of choice.

1 Introduction

Let $Y = \{y_1, \ldots, y_n\} \subset \mathbb{R}^d$ be a set of objective vectors (to be minimized) and let $r \in \mathbb{R}^d$ be a reference point. The *hypervolume indicator* of Y is

$$\mathrm{HV}(Y; r) \;=\; \mathrm{vol}\Big(\bigcup_{i=1}^{n} [y_i, r]\Big), \qquad [y_i, r] = \{x \in \mathbb{R}^d : y_i \le x \le r\}, \tag{1}$$

the volume of the union of the boxes spanned by the points and the reference point. It is the most prominent set-quality indicator in evolutionary multiobjective optimization; it is strictly monotonic with respect to Pareto dominance and is used both for performance assessment and for selection in indicator-based algorithms such as SMS-EMOA [7, 8, 11].

Equation (1) exhibits the hypervolume indicator as a special case of *Klee's measure problem* (KMP) [1]: given n axis-parallel boxes in $\mathbb{R}^d$, compute the volume of their union. The special structure is that all boxes share the corner r; inside the bounding domain each box is a *grounded orthant* $\{x : x \ge y_i\}$. Computing the hypervolume indicator is #P-hard in $n{+}d$ [9], so for constant d the relevant question is the exponent of n.

For general KMP, Overmars and Yap's classical algorithm runs in $\mathcal{O}(n^{d/2}\log n)$ time [2]; Chan improved this by an iterated-logarithmic factor [3] and finally, in the paper this implementation is based on, to a clean $\mathcal{O}(n^{d/2})$ with a strikingly simple algorithm [4]. For the orthant special case, Bringmann's $\mathcal{O}(n^{(d+2)/3})$ algorithm [5] was the first to break the $d/2$

barrier; Yıldız and Suri gave $\mathcal{O}(n^{(d-1)/2}\log n)$ for grounded boxes [6]; and Chan's Section 4 achieves the current record,

$$\mathcal{O}(n^{d/3}\operatorname{polylog} n) \qquad (d \geq 4), \tag{2}$$

explicitly naming the hypervolume indicator problem as an application. There is also good reason to believe the $d/2$ bound for general boxes is essentially optimal: deciding coverage is W[1]-hard in d, and beating $n^{d/2}$ combinatorially would improve long-standing bounds for d-clique [4].

Contribution. This paper documents two self-contained Python implementations (standard library only):

- `chan_hypervolume.py` — Chan's Section-2 algorithm for arbitrary boxes ($\mathcal{O}(n^{d/2})$ in the paper; $\mathcal{O}(n^{d/2}\log n)$ as implemented, see Section 4.1), with a hypervolume front end. To our knowledge this is among the first faithful public implementations of the FOCS 2013 algorithm.
- `chan_orthant_dby3.py` — Chan's Section-4.2 orthant algorithm for orthants of arbitrary orientation (the hypervolume case, grounded orthants, is the default front end), including the full symbolic "basic function" machinery of the paper's Lemmas 4.4 and 4.5. We are not aware of any previous implementation of this algorithm.

Section 2 walks through the simple algorithm, Section 3 through the orthant algorithm, Section 4 gives a transparent complexity analysis of the implementations as written, and Section 5 reports validation and scaling experiments.

Conventions. Throughout, d is a constant, boxes are axis-parallel, and—following Chan—all algorithms compute the measure of the *complement* of the union within a box domain Γ (a "cell"); the union's volume is recovered as $\operatorname{vol}(\Gamma)$ minus the result. For hypervolume, minimization is assumed and the domain is $\Gamma = [\ell, r]$ with $\ell_k = \min_i(y_i)_k$; each contributing box, restricted to Γ, is the grounded orthant $\{x : x \geq y_i\}$. Dominated points may be discarded beforehand; the implementation does so by default with an $\mathcal{O}(n^2 d)$ prefilter.

2 The simple $\mathcal{O}(n^{d/2})$ algorithm

Chan's Section-2 algorithm is a k-d-tree-style divide and conquer with one twist: the input is *simplified* before each cut.

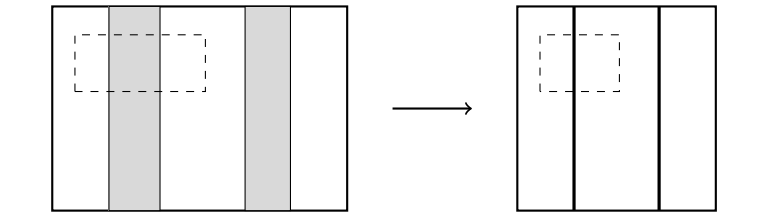
Algorithm 1 MEASURE(B, Γ) — measure of $\Gamma \setminus \bigcup B$ [4]

1: **if** $|B|$ is below a constant **then return** the answer directly (inclusion–exclusion)
2: *Simplify* B (eliminate all boxes that are slabs relative to Γ)
3: *Cut* Γ into subcells Γ_L, Γ_R by a weighted-median hyperplane
4: **return** MEASURE$(B|_{\Gamma_L}, \Gamma_L)$ + MEASURE$(B|_{\Gamma_R}, \Gamma_R)$

2.1 Step 1: simplification by slab collapse

Call a box a *slab* (relative to the cell Γ) if, restricted to Γ, it has the form $\{x : \alpha \leq x_i \leq \beta\}$, i.e. it spans Γ in every dimension except i. For each axis i, the union of the slabs of axis i is a union of disjoint intervals in the x_i-coordinate; every point inside them is covered, so they contribute nothing to the complement. Chan's elegant elimination is a *monotone re-coordination*: readjust

all x_i-values (of the cell and of every box) so that each covered interval is squeezed to length zero (Figure 1). The measure of the complement is preserved exactly, the slabs disappear, and—the key structural consequence—*every surviving box now fails to span* Γ *in at least two dimensions*, i.e. has a $(d-2)$-face intersecting Γ.



cell Γ with two x_1-slabs (shaded) after collapsing both slabs

Figure 1: Slab collapse in $d = 2$. The x_1-extents covered by slabs are squeezed to zero width by a monotone map of the x_1-axis; the dashed box is an unaffected input box whose coordinates are pushed through the same map. The uncovered area is unchanged.

The implementation (`_simplify` + `_collapse_map`) merges the slab intervals per axis in $\mathcal{O}(m \log m)$ time and pushes all coordinates through the piecewise-linear collapse map. One subtlety absent from the paper's one-paragraph description: collapsing axis i can turn a surviving box into a slab of an axis processed earlier, so the implementation repeats the sweep until no slabs remain (each extra pass removes at least one box; see Section 4.1).

2.2 Step 2: the weighted-median cut

For a $(d-2)$-face orthogonal to axes i and j, define its weight as $2^{(i+j)/d} \in (1, 4]$. The cell is cut by the hyperplane $x_1 = m$ where m is the *weighted median* of the first coordinates of the $(d-2)$-faces orthogonal to axis 1 that intersect Γ. Then the axes are renumbered cyclically, $(1, 2, \ldots, d) \mapsto (d, 1, \ldots, d-1)$: the cut axis rotates through the dimensions exactly as in a k-d tree.

Why these weights? Let N denote the total weight of all $(d-2)$-faces intersecting the cell. After one cut-plus-renumbering:

- a face orthogonal to axes $i, j \neq 1$ keeps its coordinates but its weight drops from $2^{(i+j)/d}$ to $2^{(i-1+j-1)/d}$: factor $2^{-2/d}$;
- a face orthogonal to axes $1, j$ sees its weight *rise* by $2^{(d-2)/d}$ under renumbering, but the median cut halves the total weight of such faces present in each open subcell: net factor $2^{(d-2)/d}/2 = 2^{-2/d}$.

Hence N drops by the factor $2^{2/d}$ in *each* subcell.

2.3 Analysis

Writing $T(n, N)$ for the time on inputs with n boxes and total face weight N, simplification gives $T(n, N) \leq T(\mathcal{O}(N), N) + \mathcal{O}(n)$—every surviving box owns a $(d-2)$-face of weight more than 1, so $n = \mathcal{O}(N)$ after simplification—and cutting gives $T(n, N) \leq 2\,T(n, N/2^{2/d}) + \mathcal{O}(n)$. Combining, with $T(N) = T(cN, N)$:

$$T(N) \;\leq\; 2\,T\big(N/2^{2/d}\big) + \mathcal{O}(N). \tag{3}$$

The recursion tree has 2^k nodes at level k and reaches constant size at level $k^* = \frac{d}{2}\log_2 N$, so it has $\Theta(N^{d/2})$ leaves; for $d \geq 3$ the level costs $2^k \cdot \mathcal{O}(N/2^{2k/d})$ grow geometrically (ratio $2^{1-2/d} > 1$), the leaves dominate, and $T(N) = \mathcal{O}(N^{d/2})$. Since each box has at most $4\binom{d}{2}$ faces

of weight at most 4, $N = \Theta(d^2 n)$ and the bound is $\mathcal{O}(n^{d/2})$ for constant $d \geq 3$. For $d = 2$ the two terms of (3) balance and all $\mathcal{O}(\log N)$ levels cost $\mathcal{O}(N)$: $\mathcal{O}(n \log n)$.

3 The $\mathcal{O}(n^{d/3} \operatorname{polylog} n)$ orthant algorithm

For the hypervolume indicator all boxes are grounded orthants, and Chan's Section 4.2 improves the exponent to $d/3$ by strengthening the simplification step: besides slabs, it also eliminates all boxes that are *2-sided orthants* relative to the cell—boxes spanning Γ in all but two dimensions. After this, every surviving box fails to span Γ in at least *three* dimensions, i.e. has a $(d-3)$-face intersecting Γ; cutting at weighted medians of $(d-3)$-faces (weight $2^{(i+j+k)/d}$ for a face orthogonal to axes i, j, k, with the same cyclic renumbering) makes the total face weight decay by $2^{3/d}$ per level, and the recursion tree shrinks from $\Theta(N^{d/2})$ to $\Theta(N^{d/3})$ leaves.

The price is that 2-sided orthants cannot be eliminated by re-coordination. Chan's *functional approach* absorbs them into the integrand instead.

3.1 Basic functions and staircase absorption

Definition 1 (Chan [4, Def. 4.2], adapted)**.** *A* basic function *is a finite sum of terms*

$$c \cdot [E(x)] \cdot h_1(x_1) \cdots h_d(x_d),$$

where $c \in \mathbb{R}$, each density h_i *is a univariate step function, and the* basic predicate E *is a conjunction of $\mathcal{O}(d^2)$ conditions of the form $x_j \leq f(x_i)$ or $x_j \geq f(x_i)$ with f a monotone step function. Its* complexity *is the total number of pieces of all functions involved.*

An orthant is given by its vertex v and a sign per axis: the constraint on axis k is $x_k \geq v_k$ or $x_k \leq v_k$. A constraint is *active* in the cell Γ if its hyperplane actually cuts Γ. The recursion maintains a cell Γ, a list $\bar{B}$ of surviving orthants, and a basic function F, and computes $\int_{\Gamma \setminus \bigcup \bar{B}} F(x)\, dx$; initially $F \equiv 1$. The simplification events are:

- **Cover.** An orthant with no active constraint covers Γ: the node contributes 0.
- **Slabs.** An orthant with one active constraint is a halfspace; its complement chops the cell from one side: $\Gamma.\mathrm{hi}_i \leftarrow \min(\Gamma.\mathrm{hi}_i, v_i)$ for a $\geq$-constraint, $\Gamma.\mathrm{lo}_i \leftarrow \max(\Gamma.\mathrm{lo}_i, v_i)$ for a $\leq$-constraint; opposing slabs that cross empty the cell. (For orthants no coordinate collapsing is needed—restricted to a cell, a slab is always one-sided.)
- **2-sided orthants.** The orthants active exactly in dimensions $\{i, j\}$ are quadrants in the (i, j)-plane, in four orientation classes. The complement of the union of each class is a single monotone step condition (Figure 2 shows the $(\geq, \geq)$ class):

class	covered iff	complement	shape of f
$(\geq, \geq)$	$x_j \geq \min\{b_t : a_t \leq x_i\}$	$x_j \leq f(x_i)$	decreasing
$(\leq, \leq)$	$x_j \leq \max\{b_t : a_t \geq x_i\}$	$x_j \geq f(x_i)$	decreasing
$(\geq, \leq)$	$x_j \leq \max\{b_t : a_t \leq x_i\}$	$x_j \geq f(x_i)$	increasing
$(\leq, \geq)$	$x_j \geq \min\{b_t : a_t \geq x_i\}$	$x_j \leq f(x_i)$	increasing

  These four staircases are exactly the four monotone boundary functions f^-, g^-, f^+, g^+ of Chan's Section 4.1. Absorption multiplies the class's condition into every term of F; two conditions of the same pair, sense, and monotonicity class merge by a pointwise min/max, so the predicate size stays $\mathcal{O}(d^2)$.

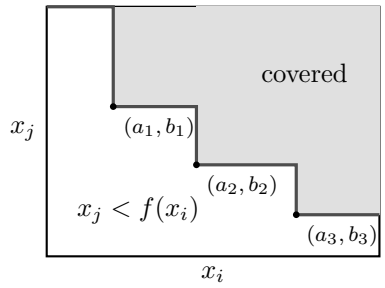


Figure 2: Absorbing 2-sided orthants of a pair (i, j). The union of the quadrants (shaded) is bounded by a staircase; the complement is described by the single condition $x_j < f(x_i)$ with f monotone decreasing—one condition per orientation class per axis pair, regardless of how many boxes were absorbed; shown is the $(\geq, \geq)$ class, the other three mirror it.

3.2 Symbolic integration (Lemma 4.4)

The engine underlying everything is one operation: integrate a basic-function term over one variable $\lambda = x_e$ between a lower bound and an upper bound. Every condition touching axis e converts into a bound on λ: $[\lambda \leq f(x_i)]$ is directly an upper bound, while $[x_j \leq f(\lambda)]$ becomes, for monotone f, a bound through the *ray preimage* $\{\lambda : f(\lambda) \geq x_j\}$, whose boundary is a monotone step function of x_j. This yields upper bounds $U_1, \ldots, U_p$ and lower bounds $V_1, \ldots, V_q$, each either a constant or a monotone step function of a single other variable. Then, with h the density of axis e and H its antiderivative,

$$\int h(\lambda) \prod_t [\lambda \leq U_t] \prod_s [\lambda \geq V_s]\, d\lambda = \Big[\max_s V_s \leq \min_t U_t\Big] \Big(H\big(\min_t U_t\big) - H\big(\max_s V_s\big)\Big).$$

The implementation performs a case split over which U_t attains the minimum and which V_s the maximum. Each guard $[U_t \leq U_{t'}]$ is again expressible in the algebra: trivially true/false for constants, a univariate indicator folded into a density when both bounds depend on the same variable, and a new condition $x_b \lesseqgtr \theta(x_a)$ with θ a composed monotone step function otherwise. Finally $H \circ$ (step bound) is again a step function, so each case contributes two new basic-function terms ($+H(U_t)$ and $-H(V_s)$): the class of basic functions is closed under integration, which is exactly Chan's Lemma 4.4. Iterating over all d axes integrates F over a box; this is the recursion's base case (combined, for up to a constant number of leftover boxes, with inclusion–exclusion over their intersections, which are boxes).

Two implementation points deserve emphasis:

- *Ties are counted exactly once.* Bounds are step functions and therefore tie on sets of positive measure (plateaus). The case split uses a first-winner rule: candidate U_t wins iff $U_t < U_{t'}$ strictly for all $t' < t$ and $U_t \leq U_{t'}$ for $t' > t$. With both strict and non-strict guards available, the split partitions the domain up to measure zero.
- *Plateau preimages are computed from pieces, never by point evaluation.* The preimage of a plateau value under a step function has positive measure; all ray preimages and cross-variable comparisons are computed exactly from piece values, and point evaluation of a step function is only ever applied to raw (Lebesgue) integration variables, where breakpoint behaviour is measure-zero. Everything else in the code may be (and is) deliberately sloppy on measure-zero sets, because every predicate is ultimately consumed by an integral.

3.3 Compression (Lemma 4.5)

Absorbing staircases makes F's complexity grow with the number of absorbed boxes, which would ruin the recurrence: the problem size passed down must be proportional to the number of *surviving* boxes. Chan's fix is to replace F by its average over the grid spanned by the coordinates of $\bar{B}$. Let X_i be the x_i-coordinates appearing among the boxes of $\bar{B}$ (plus the cell boundaries), and let $\pi_i^-(x)$ and $\pi_i^+(x)$ be the grid neighbours of x in X_i. Define

$$\tilde{F}(x) = \frac{1}{\prod_i (\pi_i^+(x_i) - \pi_i^-(x_i))} \int_{\pi_1^-(x_1)}^{\pi_1^+(x_1)} \cdots \int_{\pi_d^-(x_d)}^{\pi_d^+(x_d)} F(\lambda_1, \ldots, \lambda_d)\, d\lambda_d \cdots d\lambda_1. \tag{4}$$

By construction $\tilde{F}$ is constant on every grid cell and integrates to the same value as F over it; since $\bigcup \bar{B}$ is a union of grid cells, $\int_{\Gamma \setminus \bigcup \bar{B}} \tilde{F} = \int_{\Gamma \setminus \bigcup \bar{B}} F$. Moreover—this is the point—every occurrence of x_i in (4) is filtered through $\pi_i^{\pm}$, so every function in $\tilde{F}$ has breakpoints only on the grid: the complexity of $\tilde{F}$ is $\mathcal{O}(|\bar{B}|)$ per function, with $\mathcal{O}(1)$ terms (a constant depending on d).

The implementation computes (4) with the engine of Section 3.2 verbatim: it renames all variables of F to fresh integration variables λ_i, integrates them out one by one with the *step-function bounds* $\pi_i^-(x_i), \pi_i^+(x_i)$, and multiplies the reciprocal side lengths into the densities. A self-test verifies both the integral-preservation property and the on-grid-breakpoints property on random instances. An important soundness detail: all subsequent cuts and absorptions in the subtree use coordinates of boxes in $\bar{B}$, which lie on the grid, so every region over which $\tilde{F}$ is later integrated remains grid-aligned—the substitution is valid for the entire subtree, not just for the current node.

3.4 The recursion and its analysis

The driver mirrors Algorithm 1 with three changes: absorption handles covers, slabs, and 2-sided orthants (all cheap; run at every node); cuts use weighted medians of $(d-3)$-face coordinates with weight $2^{(i+j+k)/d}$ and cyclic renumbering, giving a per-level weight decay of $2^{3/d}$ by the same argument as in Section 2.3; and compression, which multiplies the term count by an $\mathcal{O}(1)$ factor, is *throttled*: it runs only every $\approx \log_2(r^d)$ levels. Grouping $\log_2(r^d) = d \log_2 r$ consecutive levels into one block, the face weight decays by r^3 per block, the block has $\mathcal{O}(r^d)$ subproblems, and one compression per block restores problem size $\mathcal{O}(N)$, giving Chan's recurrence

$$T(N) \;\le\; \mathcal{O}(r^d)\, T(N/r^3) + \mathcal{O}(r^d N). \tag{5}$$

Choosing $r = N^{\varepsilon}$ for a small constant $\varepsilon > 0$ makes the block depth $1/(3\varepsilon) = \mathcal{O}(1)$, so the $\mathcal{O}(1)$ compression blowup multiplies into a constant and (5) solves to $T(N) = \mathcal{O}(N^{d/3} \log^{\mathcal{O}(1)} N)$ for $d \ge 4$ [4]. (For $d = 3$ the exponent $d/3 = 1$ is not interesting: the union of orthants then has linear complexity anyway.)

3.5 Scope

The implementation covers Chan's Section 4.2 in full generality: orthants of arbitrary orientation, via the four staircase classes of Section 3.1 (the general entry point is `orthant_union_volume`; `hypervolume_dby3` is the grounded front end). The four classes are exactly the four monotone boundary functions of the union of B_{ij} in Chan's Section 4.1 [4, Sec. 4.1], and the term algebra treats them uniformly, so the generalization touches only the absorption front end, the slab handling (halfspaces now chop the cell from either side, and opposing slabs may empty it), the active-constraint tests, and the two-sided corners of the inclusion–exclusion base case. The paper's further results—the word-RAM speedups of its Section 3 and the $\mathcal{O}(n^{(d+1)/3})$ hypercube algorithm of its Section 4.3—remain out of scope.

4 Complexity of the implementations as written

4.1 `chan_hypervolume.py` (Section-2 algorithm)

Node count. The implementation performs the weighted-median cut exactly as analyzed, so the recursion tree has $\mathcal{O}(N^{d/2})$ nodes with $N = \Theta(d^2 n)$; nothing is lost here.

Per-node cost. With m boxes at a node, clipping is $\mathcal{O}(dm)$; one simplification sweep is $\mathcal{O}(d^2 m + m \log m)$ (the log from sorting slab intervals); collecting cut candidates is $\mathcal{O}(d^2 m)$; the weighted median is computed by sorting, $\mathcal{O}(m \log m)$. The implementation re-sorts at every node instead of maintaining pre-sorted coordinate lists through the recursion, and this is precisely where it loses against the paper: summing level costs as in Section 2.3 gives

$$\mathcal{O}(n^{d/2}(d^2 + \log n)) \cdot d^{\Theta(d)} \qquad \text{for } d \geq 3, \tag{6}$$

i.e. one extra $\log n$ factor (and $\mathcal{O}(n \log^2 n)$ for $d = 2$). Recovering the paper's clean $\mathcal{O}(n^{d/2})$ would require threading pre-sorted per-axis arrays through the recursion and a linear-time weighted median—both routine, neither affecting the exponent.

A caveat made explicit. The simplification sweep must be iterated because collapsing axis i can create new slabs on earlier axes. Each extra pass removes at least one box, so a node costs at most $\mathcal{O}(d^2 m^2)$ in a contrived worst case; across the tree this degrades the bound to at most $\mathcal{O}(n^{d/2+1})$. We have never observed more than two passes (one productive, one confirming) in any experiment, and the standard analysis applies whenever the pass count is $\mathcal{O}(1)$ amortized. *Space* is linear: the recursion is depth-first and box lists shrink geometrically along any root–leaf path.

4.2 `chan_orthant_dby3.py` (Section-4.2 algorithm)

Node count. Cuts follow the $(d-3)$-face weighted-median rule exactly, so the tree has $\mathcal{O}(N_{d-3}^{d/3})$ nodes, $N_{d-3} = \mathcal{O}(d^3 n)$—this is the quantity the experiments in Section 5 measure directly.

Per-node cost. Absorption is $\mathcal{O}(d^2 m \log m)$. The dominant cost is symbolic: integration and compression case-split each term into $\mathcal{O}(pq)$ new terms per eliminated axis (p, q = numbers of upper/lower bounds, both $\mathcal{O}(d)$ after per-pair condition merging), so a single compression or base-case integration multiplies term counts by a factor that is bounded for fixed d but grows like $d^{\Theta(d)}$ across the d eliminated axes—Chan's innocuous-looking "sum of $\mathcal{O}(1)$ basic functions". Three measures keep this practical: terms are merged by structural identity (summing coefficients), dead terms are pruned, and—most importantly—all functions are restricted to the current cell after every operation, which trivializes most conditions deep in the tree, where cells are small. Compression restores per-function complexity to $\mathcal{O}(|\bar{B}|)$ and is doubly throttled: at least $\max(1, \lceil 0.1\, d \log_2 n \rceil)$ levels between firings (the block schedule of Section 3.3 with $r = n^{0.1}$), *and* the representation must actually exceed a constant multiple of its post-compression size $\mathcal{O}(d^2|\bar{B}|)$. The second condition matters in practice: for grounded orthants, absorption only adds or merges conditions and never multiplies the number of terms, so compression—the sole source of term blowup—should fire only when the guarantee it provides is actually needed. (An earlier version of the implementation compressed on the level schedule alone; on one d=3, n=16 instance this compounded the per-compression blowup into $9.5 \cdot 10^4$ terms and a 150-second running time. With the size condition the same instance takes 0.01 seconds.)

The implementation therefore preserves the $\Theta(N^{d/3})$ recursion-tree bound while its per-node symbolic overhead contributes polylogarithmic factors in n with constants that grow steeply in d. This matches the character of the theoretical result: neither Bringmann's $n^{(d+2)/3}$ algorithm nor Chan's $n^{d/3}$ algorithm was designed for practical constant factors, and the experiments below confirm that the $d/2$-implementation dominates in wall-clock time at all

reachable input sizes. The value of the $d/3$-implementation is as an executable, exhaustively tested reference for the strongest known hypervolume algorithm.

5 Validation and experiments

5.1 Correctness

Both modules ship with self-tests (run the files directly). The layered oracle strategy is worth recording, since the symbolic engine is the kind of code that is easy to get subtly wrong:

1. *Property tests* for step-function operations, ray preimages (including exact plateau levels), and bound comparisons, checked pointwise against direct evaluation at thousands of random samples.
2. *Exact integration oracle*: because every function involved is a step function, $\int F$ over a box can be computed exactly by enumerating the grid of all breakpoints and sampling midpoints; the symbolic engine is required to match to 10^{-8} relative error on random basic functions in $d \leq 4$ (typical agreement is 10^{-12} or better).
3. *Compression oracle*: integral preservation of (4) over the complement of random orthant unions, plus the structural on-grid-breakpoints assertion.
4. *End-to-end*: both algorithms against $\mathcal{O}(2^n)$ inclusion–exclusion for $d \in \{3, 4, 5\}$, against each other on larger random fronts, with compression enabled and disabled, and on coordinate-tie-heavy integer grids (10 points on a 4^d grid), which exercise every degenerate branch of the tie handling. The $d/2$-implementation is additionally validated on arbitrary (non-orthant) box unions and on spherical fronts against an independent coordinate-compression enumerator. The orthant module is further checked on random *mixed-orientation* orthant unions in $d \in \{3, 4, 5\}$ (property tests of all four staircase classes, inclusion–exclusion on small instances, and the $d/2$-implementation on clipped boxes for up to $n = 80$), including forced compression.

All checks pass with relative errors at machine-precision level (worst observed: $4 \cdot 10^{-14}$).

5.2 Scaling

Test instances are *spherical fronts*: n points drawn uniformly by direction on the positive unit sphere, so that no point dominates another and nothing can be discarded a priori—the standard hard case for hypervolume algorithms. Table 1 shows the wall-clock behaviour of the $d/2$-implementation; empirical exponents are least-squares slopes in log–log scale. Real instances sit far below the worst case because simplification eliminates boxes rapidly.

Table 1: `chan_hypervolume.py` on spherical fronts (Python 3.13, one core). Exponents are empirical fits over the stated n-ranges; theoretical worst-case node exponent is $d/2$.

d	n-range	nodes $\sim n^x$	time $\sim n^x$	worst-case x	largest run
2	64–1024	1.02	1.20	1.0	n=1024: 0.07 s
3	50–800	1.04	1.31	1.5	n=800: 0.25 s
4	25–400	1.33	1.43	2.0	n=400: 0.50 s
5	25–200	1.68	1.84	2.5	n=200: 1.34 s
6	20–80	2.28	2.43	3.0	n=80: 1.51 s

Table 2 compares the two recursion trees on identical instances. The $d/3$-algorithm's tree is consistently a factor 4–5 smaller and grows with a visibly smaller exponent—the theoretical

Table 2: Recursion-tree size, $d/2$- vs. $d/3$-algorithm, same spherical-front instances (reference point $1.2 \cdot \mathbf{1}$). "exp" is the empirical node-count exponent; predicted worst-case exponents are $d/2$ and $d/3$. Wall-clock times illustrate the symbolic overhead of the $d/3$ algorithm.

		node exponent		nodes at largest n		time at largest n	
d	n-range	$d/2$-alg	$d/3$-alg	$d/2$-alg	$d/3$-alg	$d/2$-alg	$d/3$-alg
3	16–256	1.21	1.12	1267	251	0.07 s	0.13 s
4	12–96	1.70	1.49	2485	545	0.12 s	0.49 s
5	8–32	2.18	1.70	1823	393	0.08 s	0.50 s

improvement is measurable in exactly the quantity the analysis bounds. Its wall-clock time is nevertheless 2–7× higher at these sizes: the per-node work (staircase construction, condition merging, pruning, symbolic base-case integration) outweighs the smaller tree. Notably, on these instances the adaptive throttle never needs to fire a compression and the basic function stays a single term throughout; when compression is *forced* to fire (e.g. every 3 levels with a zero size threshold), results are unchanged but term counts reach the hundreds and running time grows sharply—consistent with the analysis, which permits only $\mathcal{O}(1)$ compressions per root–leaf path. Both implementations agree to at least 10^{-8} relative error on every instance.

6 Usage

```
from chan_hypervolume import hypervolume # O(n^{d/2} log n), practical
from chan_orthant_dby3 import hypervolume_dby3 # O(n^{d/3} polylog n), reference

points = [(0.2, 0.7, 0.3), (0.5, 0.2, 0.6), (0.8, 0.4, 0.1)]
ref = (1.0, 1.0, 1.0) # minimization; use maximize=True otherwise

hv = hypervolume(points, ref)
hv3 = hypervolume_dby3(points, ref) # identical value, d >= 3 only
```

Both functions accept `maximize=True` and `prefilter=False`; the general Klee's-measure entry point `union_volume(boxes)` in `chan_hypervolume.py` accepts arbitrary boxes as `(lo, hi)` corner pairs, and `orthant_union_volume(orthants, lo, hi)` in `chan_orthant_dby3.py` accepts arbitrary-orientation orthants as `(vertex, signs)` pairs measured inside a domain box. The solver classes (`ChanMeasure`, `ChanOrthantMeasure`) expose node counts and, for the latter, the compression interval and a switch to disable compression.

7 Conclusions

Chan's "easy" algorithm fully lives up to its name: the $d/2$ implementation is under 300 lines of executable logic, uses no data structure beyond sorted lists, and computes exact hypervolumes of fronts with hundreds of points in seconds up to $d = 7$. The $d/3$ orthant algorithm is a different kind of object: its recursion is just as simple, but the basic-function algebra it stands on—closure of step-function predicates under integration, and grid-averaging compression—is intricate to implement correctly, chiefly because of positive-measure ties between step functions. We believe the implementation presented here is the first of that algorithm; its recursion-tree scaling matches the $d/3$ theory, and it provides an executable specification against which faster engineered variants (or an implementation of the hypercube case) can be checked.

Code availability. Both modules, the test suites, and the benchmark scripts are available in the accompanying repository; everything depends only on the Python standard library.

References

[1] V. Klee. Can the measure of $\bigcup[a_i, b_i]$ be computed in less than $O(n \log n)$ steps? *American Mathematical Monthly*, 84:284–285, 1977.

[2] M. H. Overmars and C.-K. Yap. New upper bounds in Klee's measure problem. *SIAM Journal on Computing*, 20(6):1034–1045, 1991.

[3] T. M. Chan. A (slightly) faster algorithm for Klee's measure problem. *Computational Geometry: Theory and Applications*, 43(3):243–250, 2010.

[4] T. M. Chan. Klee's measure problem made easy. In *Proc. 54th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 410–419, 2013.

[5] K. Bringmann. An improved algorithm for Klee's measure problem on fat boxes. *Computational Geometry: Theory and Applications*, 45(5–6):225–233, 2012.

[6] H. Yıldız and S. Suri. On Klee's measure problem for grounded boxes. In *Proc. 28th ACM Symposium on Computational Geometry (SoCG)*, pages 111–120, 2012.

[7] E. Zitzler and L. Thiele. Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257–271, 1999.

[8] N. Beume, B. Naujoks, and M. Emmerich. SMS-EMOA: Multiobjective selection based on dominated hypervolume. *European Journal of Operational Research*, 181(3):1653–1669, 2007.

[9] N. Beume, C. M. Fonseca, M. López-Ibáñez, L. Paquete, and J. Vahrenhold. On the complexity of computing the hypervolume indicator. *IEEE Transactions on Evolutionary Computation*, 13(5):1075–1082, 2009.

[10] L. While, P. Hingston, L. Barone, and S. Huband. A faster algorithm for calculating hypervolume. *IEEE Transactions on Evolutionary Computation*, 10(1):29–38, 2006.

[11] A. P. Guerreiro, C. M. Fonseca, and L. Paquete. The hypervolume indicator: Computational problems and algorithms. *ACM Computing Surveys*, 54(6):119:1–119:42, 2021.

[12] M. S. Paterson and F. F. Yao. Optimal binary space partitions for orthogonal objects. *Journal of Algorithms*, 13(1):99–113, 1992.